

Exercício 1: LojaVirtual

Enunciado:

1. Crie uma base de dados chamada LojaVirtual.
2. Dentro dessa base, crie uma tabela chamada Clientes, com os seguintes campos:
 - id – inteiro, chave primária, gerado automaticamente
 - nome – texto, até 100 caracteres
 - email – texto, até 150 caracteres
3. Insira **3 clientes** na tabela Clientes, com nomes e e-mails fictícios.
4. Exclua a tabela Clientes.
5. Exclua a base de dados LojaVirtual.

Resolução:

```
-- 1. Criar a base de dados
CREATE DATABASE LojaVirtual;

-- 2. Selecionar a base
USE LojaVirtual;

-- 3. Criar a tabela Clientes
CREATE TABLE Clientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100),
  email VARCHAR(150)
);

-- 4. Inserir 3 clientes
INSERT INTO Clientes (nome, email) VALUES
  ('João Silva', 'joao.silva@example.com'),
  ('Maria Souza', 'maria.souza@example.com'),
  ('Carlos Lima', 'carlos.lima@example.com');

-- 5. Excluir a tabela Clientes
DROP TABLE Clientes;

-- 6. Excluir a base de dados LojaVirtual
DROP DATABASE LojaVirtual;
```

Exercício 2: EscolaDB

Enunciado:

1. Crie uma base de dados chamada EscolaDB.
2. Dentro dessa base, crie uma tabela chamada Alunos, com os campos:
 - matricula – inteiro, chave primária
 - nome – texto, até 80 caracteres

- serie – texto, até 20 caracteres (por exemplo: "6º ano", "9º ano")
- 3. Insira pelo menos **4 alunos** na tabela Alunos.
- 4. Exclua a tabela Alunos.
- 5. Exclua a base de dados EscolaDB.

Resolução:

```
-- 1. Criar a base de dados
CREATE DATABASE EscolaDB;

-- 2. Selecionar a base
USE EscolaDB;

-- 3. Criar a tabela Alunos
CREATE TABLE Alunos (
    matricula INT PRIMARY KEY,
    nome VARCHAR(80),
    serie VARCHAR(20)
);

-- 4. Inserir 4 alunos
INSERT INTO Alunos (matricula, nome, serie) VALUES
    (1, 'Ana Paula', '6º ano'),
    (2, 'Bruno Santos', '7º ano'),
    (3, 'Carla Souza', '9º ano'),
    (4, 'Diego Lima', '8º ano');

-- 5. Excluir a tabela Alunos
DROP TABLE Alunos;

-- 6. Excluir a base de dados EscolaDB
DROP DATABASE EscolaDB;
```

Exercício 3: BibliotecaDB

Enunciado:

1. Crie uma base de dados chamada BibliotecaDB.
2. Dentro dessa base, crie uma tabela chamada Livros, com os campos:
 - id_livro – inteiro, chave primária
 - titulo – texto, até 150 caracteres
 - autor – texto, até 100 caracteres
3. Insira **5 livros** na tabela Livros, com título e autor fictícios.
4. Exclua a tabela Livros.
5. Exclua a base de dados BibliotecaDB.

Resolução:

```
-- 1. Criar a base de dados
```

```

CREATE DATABASE BibliotecaDB;

-- 2. Selecionar a base
USE BibliotecaDB;

-- 3. Criar a tabela Livros
CREATE TABLE Livros (
    id_livro INT PRIMARY KEY,
    titulo VARCHAR(150),
    autor VARCHAR(100)
);

-- 4. Inserir 5 livros
INSERT INTO Livros (id_livro, titulo, autor) VALUES
    (1, 'Introdução ao MySQL', 'João Pereira'),
    (2, 'Algoritmos em C', 'Maria Fernandes'),
    (3, 'Banco de Dados Relacionais', 'Carlos Silva'),
    (4, 'Programação em Python', 'Ana Souza'),
    (5, 'Lógica de Programação', 'Paulo Santos');

-- 5. Excluir a tabela Livros
DROP TABLE Livros;

-- 6. Excluir a base de dados BibliotecaDB
DROP DATABASE BibliotecaDB;

```

Exercício 4: RH_Empresa

Enunciado:

1. Crie uma base de dados chamada RH_Empresa.
2. Dentro dessa base, crie uma tabela chamada Funcionarios, com os campos:
 - id_funcionario – inteiro, chave primária
 - nome – texto, até 100 caracteres
 - cargo – texto, até 50 caracteres
3. Insira **3 funcionários** na tabela Funcionarios.
4. Exclua a tabela Funcionarios.
5. Exclua a base de dados RH_Empresa.

Resolução:

```

-- 1. Criar a base de dados
CREATE DATABASE RH_Empresa;

-- 2. Selecionar a base
USE RH_Empresa;

-- 3. Criar a tabela Funcionarios
CREATE TABLE Funcionarios (
    id_funcionario INT PRIMARY KEY,

```

```

    nome VARCHAR(100),
    cargo VARCHAR(50)
);

-- 4. Inserir 3 funcionários
INSERT INTO Funcionarios (id_funcionario, nome, cargo) VALUES
    (1, 'Fernanda Costa', 'Analista de RH'),
    (2, 'Rafael Souza', 'Assistente Administrativo'),
    (3, 'Lucas Pereira', 'Gerente de Projetos');

-- 5. Excluir a tabela Funcionarios
DROP TABLE Funcionarios;

-- 6. Excluir a base de dados RH_Empresa
DROP DATABASE RH_Empresa;

```

Exercício 5: VendasDB

Enunciado:

1. Crie uma base de dados chamada VendasDB.
2. Dentro dessa base, crie uma tabela chamada Produtos, com os campos:
 - id_produto – inteiro, chave primária
 - nome_produto – texto, até 100 caracteres
 - preco – número decimal
3. Insira **4 produtos** na tabela Produtos.
4. Exclua a tabela Produtos.
5. Exclua a base de dados VendasDB.

Resolução:

```

-- 1. Criar a base de dados
CREATE DATABASE VendasDB;

-- 2. Selecionar a base
USE VendasDB;

-- 3. Criar a tabela Produtos
CREATE TABLE Produtos (
    id_produto INT PRIMARY KEY,
    nome_produto VARCHAR(100),
    preco DECIMAL(10,2)
);

-- 4. Inserir 4 produtos
INSERT INTO Produtos (id_produto, nome_produto, preco) VALUES
    (1, 'Teclado USB', 99.90),
    (2, 'Mouse Óptico', 59.50),
    (3, 'Monitor 24 Polegadas', 899.00),
    (4, 'Headset Gamer', 199.99);

```

-- 5. Excluir a tabela Produtos
DROP TABLE Produtos;

-- 6. Excluir a base de dados VendasDB
DROP DATABASE VendasDB;

Exercício 6: CursosOnline

Enunciado:

1. Crie uma base de dados chamada CursosOnline.
2. Dentro dessa base, crie uma tabela chamada Cursos, com os campos:
 - id_curso – inteiro, chave primária
 - nome_curso – texto, até 120 caracteres
 - carga_horaria – inteiro (horas)
3. Insira **3 cursos** na tabela Cursos.
4. Exclua a tabela Cursos.
5. Exclua a base de dados CursosOnline.

Resolução:

-- 1. Criar a base de dados
CREATE DATABASE CursosOnline;

-- 2. Selecionar a base
USE CursosOnline;

-- 3. Criar a tabela Cursos
CREATE TABLE Cursos (
 id_curso INT PRIMARY KEY,
 nome_curso VARCHAR(120),
 carga_horaria INT
);

-- 4. Inserir 3 cursos
INSERT INTO Cursos (id_curso, nome_curso, carga_horaria) VALUES
 (1, 'Introdução à Programação', 40),
 (2, 'Desenvolvimento Web', 60),
 (3, 'Banco de Dados Relacionais', 50);

-- 5. Excluir a tabela Cursos
DROP TABLE Cursos;

-- 6. Excluir a base de dados CursosOnline
DROP DATABASE CursosOnline;

Exercício 7: CinemaDB

Enunciado:

1. Crie uma base de dados chamada CinemaDB.
2. Dentro dessa base, crie uma tabela chamada Filmes, com os campos:
 - id_filme – inteiro, chave primária
 - titulo – texto, até 150 caracteres
 - ano_lancamento – inteiro (ano)
3. Insira **5 filmes** na tabela Filmes.
4. Exclua a tabela Filmes.
5. Exclua a base de dados CinemaDB.

Resolução:

```
-- 1. Criar a base de dados
CREATE DATABASE CinemaDB;

-- 2. Selecionar a base
USE CinemaDB;

-- 3. Criar a tabela Filmes
CREATE TABLE Filmes (
  id_filme INT PRIMARY KEY,
  titulo VARCHAR(150),
  ano_lancamento INT
);

-- 4. Inserir 5 filmes
INSERT INTO Filmes (id_filme, titulo, ano_lancamento) VALUES
(1, 'A Aventura Começa', 2010),
(2, 'Noite Estrelada', 2015),
(3, 'Caminho Sem Volta', 2018),
(4, 'Heróis do Amanhã', 2020),
(5, 'O Retorno do Tempo', 2022);

-- 5. Excluir a tabela Filmes
DROP TABLE Filmes;

-- 6. Excluir a base de dados CinemaDB
DROP DATABASE CinemaDB;
```

Exercício 8: SuporteDB

Enunciado:

1. Crie uma base de dados chamada SuporteDB.
2. Dentro dessa base, crie uma tabela chamada Chamados, com os campos:
 - id_chamado – inteiro, chave primária
 - titulo – texto, até 100 caracteres
 - status – texto, até 20 caracteres (por exemplo: "aberto", "fechado")

3. Insira **3 chamados** na tabela Chamados.
4. Exclua a tabela Chamados.
5. Exclua a base de dados SuporteDB.

Resolução:

```
-- 1. Criar a base de dados
CREATE DATABASE SuporteDB;

-- 2. Selecionar a base
USE SuporteDB;

-- 3. Criar a tabela Chamados
CREATE TABLE Chamados (
    id_chamado INT PRIMARY KEY,
    titulo VARCHAR(100),
    status VARCHAR(20)
);

-- 4. Inserir 3 chamados
INSERT INTO Chamados (id_chamado, titulo, status) VALUES
    (1, 'Erro ao acessar o sistema', 'aberto'),
    (2, 'Atualização de cadastro', 'fechado'),
    (3, 'Dúvida sobre relatório', 'aberto');

-- 5. Excluir a tabela Chamados
DROP TABLE Chamados;

-- 6. Excluir a base de dados SuporteDB
DROP DATABASE SuporteDB;
```

Exercício 9: AcademiaDB

Enunciado:

1. Crie uma base de dados chamada AcademiaDB.
2. Dentro dessa base, crie uma tabela chamada AlunosAcademia, com os campos:
 - id_aluno – inteiro, chave primária
 - nome – texto, até 100 caracteres
 - plano – texto, até 50 caracteres (por exemplo: "mensal", "anual")
3. Insira **4 alunos** na tabela AlunosAcademia.
4. Exclua a tabela AlunosAcademia.
5. Exclua a base de dados AcademiaDB.

Resolução:

```
-- 1. Criar a base de dados
CREATE DATABASE AcademiaDB;

-- 2. Selecionar a base
```

```

USE AcademiaDB;

-- 3. Criar a tabela AlunosAcademia
CREATE TABLE AlunosAcademia (
    id_aluno INT PRIMARY KEY,
    nome VARCHAR(100),
    plano VARCHAR(50)
);

-- 4. Inserir 4 alunos
INSERT INTO AlunosAcademia (id_aluno, nome, plano) VALUES
    (1, 'João Mendes', 'mensal'),
    (2, 'Laura Silva', 'anual'),
    (3, 'Pedro Costa', 'mensal'),
    (4, 'Marina Souza', 'trimestral');

-- 5. Excluir a tabela AlunosAcademia
DROP TABLE AlunosAcademia;

-- 6. Excluir a base de dados AcademiaDB
DROP DATABASE AcademiaDB;

```

Exercício 10: EventosDB

Enunciado:

1. Crie uma base de dados chamada EventosDB.
2. Dentro dessa base, crie uma tabela chamada Eventos, com os campos:
 - id_evento – inteiro, chave primária
 - nome_evento – texto, até 120 caracteres
 - cidade – texto, até 80 caracteres
3. Insira **3 eventos** na tabela Eventos.
4. Exclua a tabela Eventos.
5. Exclua a base de dados EventosDB.

Resolução:

```

-- 1. Criar a base de dados
CREATE DATABASE EventosDB;

-- 2. Selecionar a base
USE EventosDB;

-- 3. Criar a tabela Eventos
CREATE TABLE Eventos (
    id_evento INT PRIMARY KEY,
    nome_evento VARCHAR(120),
    cidade VARCHAR(80)
);

```


-- 4. Inserir 3 eventos

```
INSERT INTO Eventos (id_evento, nome_evento, cidade) VALUES  
  (1, 'Conferência de Tecnologia', 'São Paulo'),  
  (2, 'Feira de Negócios', 'Rio de Janeiro'),  
  (3, 'Workshop de Programação', 'Belo Horizonte');
```

-- 5. Excluir a tabela Eventos

```
DROP TABLE Eventos;
```

-- 6. Excluir a base de dados EventosDB

```
DROP DATABASE EventosDB;
```

Select

```
-- 1. CRIAR O BANCO DE DADOS
CREATE DATABASE escola;

-- Selecionar o banco
USE escola;

-- 2. CRIAR A TABELA
CREATE TABLE alunos (
  id INT PRIMARY KEY,
  nome VARCHAR(50),
  idade INT,
  cidade VARCHAR(50),
  nota INT
);

-- 3. INSERIR 20 REGISTROS
INSERT INTO alunos (id, nome, idade, cidade, nota) VALUES
(1, 'Ana', 20, 'São Paulo', 85),
(2, 'Bruno', 22, 'Rio de Janeiro', 70),
(3, 'Carla', 19, 'São Paulo', 95),
(4, 'Daniel', 21, 'Belo Horizonte', 60),
(5, 'Elisa', 23, 'Rio de Janeiro', 88),
(6, 'Fernando', 20, 'São Paulo', 75),
(7, 'Gabriela', 22, 'Curitiba', 90),
(8, 'Henrique', 18, 'São Paulo', 65),
(9, 'Isabela', 24, 'Salvador', 92),
(10, 'João', 21, 'Rio de Janeiro', 78),
(11, 'Karina', 19, 'Belo Horizonte', 82),
(12, 'Lucas', 23, 'Curitiba', 55),
(13, 'Marina', 20, 'Salvador', 97),
(14, 'Nicolas', 22, 'São Paulo', 73),
(15, 'Olivia', 18, 'Rio de Janeiro', 80),
(16, 'Pedro', 25, 'Belo Horizonte', 68),
(17, 'Quintino', 19, 'Curitiba', 91),
(18, 'Rafaela', 24, 'São Paulo', 76),
(19, 'Sofia', 21, 'Salvador', 84),
(20, 'Thiago', 20, 'Rio de Janeiro', 89);

-- CONSULTAS DE VERIFICAÇÃO
SELECT * FROM alunos;
```

Se preferir inserir um registro por vez:

```
INSERT INTO alunos (id, nome, idade, cidade, nota)
VALUES (21, 'Ursula', 22, 'São Paulo', 87);
```

Comando SELECT em SQL

SELECT * FROM : Todos os campos

Retorna **todas as colunas** da tabela.

```
SELECT * FROM alunos;
```

- Use SELECT * apenas para consultas rápidas. Em produção, prefira selecionar apenas os campos necessários.

SELECT com campos específicos

Retorna **apenas as colunas que você escolher**.

```
SELECT nome, nota FROM alunos;  
SELECT nome, idade, cidade FROM alunos;
```

SELECT com WHERE: Filtro simples

Filtra linhas com base em **uma condição**.

```
SELECT * FROM alunos WHERE cidade = 'São Paulo';  
SELECT nome, nota FROM alunos WHERE nota >= 85;
```

SELECT com WHERE e AND: Duas condições (ambas verdadeiras)

O AND exige que **todas** as condições sejam verdadeiras.

```
SELECT * FROM alunos WHERE cidade = 'São Paulo' AND nota >= 80;  
SELECT nome, idade FROM alunos WHERE idade >= 20 AND idade <= 22;
```

SELECT com OR: Pelo menos uma condição verdadeira

O OR retorna linhas onde **qualquer uma** das condições for verdadeira.

```
SELECT * FROM alunos WHERE cidade = 'Curitiba' OR cidade = 'Belo Horizonte';  
SELECT nome, nota FROM alunos WHERE nota < 70 OR nota > 90;
```

SELECT com AND e OR combinados: Parênteses para prioridade

Quando misturamos AND e OR, usamos **parênteses** para definir a ordem de avaliação, assim como na matemática.

```
SELECT * FROM alunos
  WHERE
    (cidade = 'São Paulo' OR cidade = 'Rio de Janeiro')
  AND
    nota >= 75;
```

```
SELECT nome, idade, cidade FROM alunos
  WHERE
    idade = 20
  AND
    (cidade = 'São Paulo' OR cidade = 'Curitiba');
```

SELECT com ORDER BY: Ordenação

ASC: Crescente (padrão)

```
SELECT * FROM alunos ORDER BY nota ASC;
```

DESC: Decrescente

```
SELECT nome, nota FROM alunos ORDER BY nota DESC;
```

ORDER BY com múltiplos campos

```
SELECT * FROM alunos ORDER BY cidade ASC, nota DESC;
Primeiro ordena por cidade (A-Z), depois por nota (maior para menor).
```

Combinações finais: Tudo junto

```
SELECT nome, cidade, nota FROM alunos
  WHERE
    (cidade = 'São Paulo' OR cidade = 'Rio de Janeiro')
  AND
    nota >= 70 ORDER BY nota DESC;
```

```
SELECT nome, idade, nota FROM
alunos WHERE
idade BETWEEN 20 AND 22
AND nota > 80 ORDER BY idade ASC,
nota DESC;
```

Observação: A ordem das cláusulas é sempre esta: **SELECT → FROM → WHERE → ORDER BY.**

Exercícios de SQL: Comando SELECT

Tabela: livros

Crie um banco de dados chamado biblioteca, uma tabela livros com os campos abaixo, insira os registros e depois resolva os SELECTs.

Campos da tabela:

- id INT
- titulo VARCHAR(80)
- autor VARCHAR(60)
- genero VARCHAR(40)
- ano INT
- paginas INT

Registros para inserir:

id	titulo	autor	genero	ano	paginas
1	Dom Casmurro	Machado de Assis	Romance	1899	256
2	1984	George Orwell	Distopia	1949	328
3	O Pequeno Príncipe	Antoine Saint-Exupéry	Infantil	1943	96
4	A Revolução dos Bichos	George Orwell	Sátira	1945	152
5	Grande Sertão: Veredas	Guimarães Rosa	Romance	1956	624
6	O Hobbit	J.R.R. Tolkien	Fantasia	1937	310
7	Capitães da Areia	Jorge Amado	Romance	1937	280
8	A Metamorfose	Franz Kafka	Ficção	1915	80

Perguntas (SELECTs):

1. Selecione todos os campos de todos os livros.
2. Selecione apenas o título e o autor dos livros do gênero "Romance".
3. Selecione título, ano e páginas dos livros com mais de 200 páginas, ordenados do mais recente para o mais antigo.

Resolução:

-- 1. Todos os campos de todos os livros

```
SELECT * FROM livros;
```

-- 2. Título e autor dos livros do gênero Romance

```
SELECT titulo, autor FROM livros WHERE genero = 'Romance';
```

-- 3. Título, ano e páginas com mais de 200 páginas, ordenados por ano DESC

```
SELECT titulo, ano, paginas FROM livros  
WHERE paginas > 200  
ORDER BY ano DESC;
```

Tabela: funcionarios

Crie um banco empresa, tabela funcionarios, insira os dados e resolva os SELECTs.

Campos:

- id INT
- nome VARCHAR(60)
- cargo VARCHAR(50)
- departamento VARCHAR(40)
- salario INT
- cidade VARCHAR(50)

Registros:

id	nome	cargo	departamento	salario	cidade
1	Carlos Silva	Analista	TI	5500	São Paulo
2	Marina Costa	Gerente	RH	8200	Rio de Janeiro
3	Felipe Souza	Desenvolvedor	TI	6200	São Paulo
4	Amanda Lima	Analista	Financeiro	5100	Belo Horizonte
5	Roberto Alves	Assistente	RH	3200	Curitiba
6	Juliana Torres	Desenvolvedor	TI	6800	São Paulo
7	Eduardo Rocha	Gerente	Financeiro	9000	Rio de Janeiro
8	Patrícia Dias	Analista	RH	4800	Curitiba

Perguntas (SELECTs):

1. Selecione nome, cargo e salário de todos os funcionários do departamento de TI.
2. Selecione todos os funcionários que ganham acima de 5000 E moram em São Paulo.
3. Selecione nome, departamento e salário dos funcionários do RH OU do Financeiro, ordenados por salário do maior para o menor.

Resolução:

-- 1. Funcionários do departamento de TI

```
SELECT nome, cargo, salario FROM funcionarios WHERE departamento = 'TI';
```

-- 2. Salário acima de 5000 E cidade = São Paulo

```
SELECT * FROM funcionarios  
WHERE salario > 5000 AND cidade = 'São Paulo';
```

-- 3. RH ou Financeiro, ordenados por salário DESC

```
SELECT nome, departamento, salario FROM funcionarios  
WHERE departamento = 'RH' OR departamento = 'Financeiro'  
ORDER BY salario DESC;
```

Tabela: produtos

Crie um banco loja, tabela produtos, insira os dados e resolva os SELECTs.

Campos:

- id INT
- nome VARCHAR(60)
- categoria VARCHAR(40)
- marca VARCHAR(40)
- preco INT
- estoque INT

Registros:

id	nome	categoria	marca	preco	estoque
1	Notebook Ultra	Informática	TechPlus	4500	15
2	Smartphone Z10	Celular	MobileX	3200	30
3	Fone Bluetooth	Acessório	SoundMax	250	100
4	Monitor 27"	Informática	ViewPro	1800	20
5	Teclado Mecânico	Acessório	KeyTech	450	50
6	Tablet T8	Informática	TechPlus	2100	12
7	Carregador USB	Acessório	MobileX	80	200
8	Smartwatch W2	Celular	SoundMax	1200	25

Perguntas (SELECTs):

1. Selecione nome, marca e preço de todos os produtos da categoria "Informática".
2. Selecione todos os produtos da marca "TechPlus" OU da marca "MobileX".
3. Selecione nome, categoria e estoque dos produtos com preço acima de 1000 E estoque menor que 30, ordenados por preço crescente.

Resolução:

-- 1. Produtos da categoria Informática

```
SELECT nome, marca, preco FROM produtos WHERE categoria = 'Informática';
```

-- 2. Produtos da marca TechPlus OU MobileX

```
SELECT * FROM produtos  
WHERE marca = 'TechPlus' OR marca = 'MobileX';
```

-- 3. Preço > 1000 E estoque < 30, ordenado por preço ASC

```
SELECT nome, categoria, estoque FROM produtos  
WHERE preco > 1000 AND estoque < 30  
ORDER BY preco ASC;
```

Tabela: filmes

Crie um banco cinema, tabela filmes, insira os dados e resolva os SELECTs.

Campos:

- id INT
- titulo VARCHAR(80)
- diretor VARCHAR(60)
- genero VARCHAR(40)
- ano INT
- duracao INT

Registros:

id	titulo	diretor	genero	ano	duracao
1	Matrix	Wachowski	Ficção	1999	136
2	O Poderoso Chefão	Coppola	Drama	1972	175
3	Toy Story	Lasseter	Animação	1995	81
4	Interestelar	Nolan	Ficção	2014	169
5	Clube da Luta	Fincher	Drama	1999	139
6	Frozen	Buck	Animação	2013	102
7	Parasita	Bong Joon-ho	Drama	2019	132
8	Mad Max	Miller	Ação	2015	120

Perguntas (SELECTs):

1. Selecione título e ano de todos os filmes de Ficção ou Ação.
2. Selecione todos os filmes com duração maior que 130 minutos E ano maior que 2000.
3. Selecione título, diretor e duração dos filmes de Drama, ordenados por duração do menor para o maior.

Resolução:

-- 1. Filmes de Ficção ou Ação

```
SELECT titulo, ano FROM filmes  
WHERE genero = 'Ficção' OR genero = 'Ação';
```

-- 2. Duração > 130 E ano > 2000

```
SELECT * FROM filmes  
WHERE duracao > 130 AND ano > 2000;
```

-- 3. Filmes de Drama ordenados por duração ASC

```
SELECT titulo, diretor, duracao FROM filmes  
WHERE genero = 'Drama'  
ORDER BY duracao ASC;
```


Tabela: carros

Crie um banco concessionaria, tabela carros, insira os dados e resolva os SELECTs.

Campos:

- id INT
- modelo VARCHAR(60)
- marca VARCHAR(40)
- cor VARCHAR(30)
- ano INT
- preco INT

Registros:

id	modelo	marca	cor	ano	preco
1	Civic	Honda	Preto	2022	125000
2	Corolla	Toyota	Branco	2023	135000
3	Onix	Chevrolet	Prata	2021	75000
4	Gol	Volkswagen	Vermelho	2020	62000
5	HB20	Hyundai	Azul	2022	78000
6	Compass	Jeep	Preto	2023	165000
7	Kwid	Renault	Branco	2021	55000
8	Polo	Volkswagen	Prata	2022	85000

Perguntas (SELECTs):

1. Selecione modelo, marca e preço de todos os carros da marca Volkswagen.
2. Selecione todos os carros com preço acima de 100000 E ano igual a 2023.
3. Selecione modelo, cor e ano dos carros da cor Preto OU da cor Branco, ordenados por ano do mais novo para o mais velho.

Resolução:

-- 1. Carros da marca Volkswagen

```
SELECT modelo, marca, preco FROM carros WHERE marca = 'Volkswagen';
```

-- 2. Preço > 100000 E ano = 2023

```
SELECT * FROM carros  
WHERE preco > 100000 AND ano = 2023;
```

-- 3. Cor Preto OU Branco, ordenados por ano DESC

```
SELECT modelo, cor, ano FROM carros  
WHERE cor = 'Preto' OR cor = 'Branco'  
ORDER BY ano DESC;
```

Tabela: cidades

Crie um banco geografia, tabela cidades, insira os dados e resolva os SELECTs.

Campos:

- id INT
- nome VARCHAR(60)
- estado VARCHAR(40)
- regio VARCHAR(30)
- populacao INT
- fundacao INT

Registros:

id	nome	estado	regiao	populacao	fundacao
1	São Paulo	SP	Sudeste	12300000	1554
2	Rio de Janeiro	RJ	Sudeste	6748000	1565
3	Salvador	BA	Nordeste	2887000	1549
4	Brasília	DF	Centro-Oeste	3055000	1960
5	Fortaleza	CE	Nordeste	2686000	1726
6	Curitiba	PR	Sul	1949000	1693
7	Manaus	AM	Norte	2219000	1669
8	Porto Alegre	RS	Sul	1488000	1772

Perguntas (SELECTs):

1. Selecione nome e estado de todas as cidades da região Sudeste.
2. Selecione todas as cidades com população acima de 3 milhões E fundação anterior ao ano 1700.
3. Selecione nome, região e população das cidades do Nordeste OU do Norte, ordenadas por população da maior para a menor.

Resolução:

-- 1. Cidades da região Sudeste

```
SELECT nome, estado FROM cidades WHERE regio = 'Sudeste';
```

-- 2. População > 3000000 E fundação < 1700

```
SELECT * FROM cidades  
WHERE populacao > 3000000 AND fundacao < 1700;
```

-- 3. Nordeste OU Norte, ordenadas por população DESC

```
SELECT nome, regio, populacao FROM cidades  
WHERE regio = 'Nordeste' OR regio = 'Norte'  
ORDER BY populacao DESC;
```

Tabela: jogadores

Crie um banco esporte, tabela jogadores, insira os dados e resolva os SELECTs.

Campos:

- id INT
- nome VARCHAR(60)
- time VARCHAR(50)
- posicao VARCHAR(40)
- idade INT
- gols INT

Registros:

id	nome	time	posicao	idade	gols
1	Neymar	Santos	Atacante	32	79
2	Richarlison	Flamengo	Atacante	27	45
3	Casemiro	São Paulo	Volante	32	12
4	Vinícius Jr	Real Madrid	Atacante	24	68
5	Marquinhos	PSG	Zagueiro	30	8
6	Alisson	Liverpool	Goleiro	31	0
7	Rodrygo	Real Madrid	Atacante	23	35
8	Bruno Guimarães	Newcastle	Meio-campo	26	15

Perguntas (SELECTs):

1. Selecione nome, time e posição de todos os jogadores que são Atacantes.
2. Selecione todos os jogadores com idade menor que 30 E gols maior que 30.
3. Selecione nome, time e gols dos jogadores do Real Madrid OU do Flamengo, ordenados por gols do maior para o menor.

Resolução:

-- 1. Jogadores que são Atacantes

```
SELECT nome, time, posicao FROM jogadores WHERE posicao = 'Atacante';
```

-- 2. Idade < 30 E gols > 30

```
SELECT * FROM jogadores  
WHERE idade < 30 AND gols > 30;
```

-- 3. Real Madrid OU Flamengo, ordenados por gols DESC

```
SELECT nome, time, gols FROM jogadores  
WHERE time = 'Real Madrid' OR time = 'Flamengo'  
ORDER BY gols DESC;
```

Tabela: receitas

Crie um banco culinaria, tabela receitas, insira os dados e resolva os SELECTs.

Campos:

- id INT
- nome VARCHAR(80)
- categoria VARCHAR(40)
- dificuldade VARCHAR(30)
- tempo INT
- calorias INT

Registros:

id	nome	categoria	dificuldade	tempo	calorias
1	Bolo de Cenoura	Doce	Fácil	50	350
2	Feijoada	Salgada	Difícil	180	520
3	Salada Caesar	Salada	Fácil	15	280
4	Lasanha	Massa	Médio	90	450
5	Pudim	Doce	Médio	60	380
6	Sopa de Legumes	Salgada	Fácil	40	180
7	Strogonoff	Salgada	Médio	45	420
8	Torta de Limão	Doce	Médio	70	310

Perguntas (SELECTs):

1. Selecione nome e tempo de todas as receitas da categoria Doce.
2. Selecione todas as receitas com dificuldade Fácil E tempo menor que 60 minutos.
3. Selecione nome, categoria e calorias das receitas da categoria Salgada OU Massa, ordenadas por calorias da menor para a maior.

Resolução:

-- 1. Receitas da categoria Doce

```
SELECT nome, tempo FROM receitas WHERE categoria = 'Doce';
```

-- 2. Dificuldade Fácil E tempo < 60

```
SELECT * FROM receitas  
WHERE dificuldade = 'Fácil' AND tempo < 60;
```

-- 3. Salgada OU Massa, ordenadas por calorias ASC

```
SELECT nome, categoria, calorias FROM receitas  
WHERE categoria = 'Salgada' OR categoria = 'Massa'  
ORDER BY calorias ASC;
```

Tabela: músicas

Crie um banco musica, tabela musicas, insira os dados e resolva os SELECTs.

Campos:

- id INT
- titulo VARCHAR(80)
- artista VARCHAR(60)
- genero VARCHAR(40)
- ano INT
- duracao INT

Registros:

id	titulo	artista	genero	ano	duracao
1	Bohemian Rhapsody	Queen	Rock	1975	354
2	Garota de Ipanema	Tom Jobim	Bossa Nova	1962	185
3	Smells Like Teen Spirit	Nirvana	Rock	1991	301
4	Águas de Março	Tom Jobim	Bossa Nova	1972	198
5	Billie Jean	Michael Jackson	Pop	1983	294
6	Trem das Onze	Adoniran Barbosa	Samba	1964	192
7	Shape of You	Ed Sheeran	Pop	2017	234
8	Aquarela do Brasil	Ary Barroso	Samba	1939	210

Perguntas (SELECTs):

1. Selecione título e artista de todas as músicas do gênero Rock.
2. Selecione todas as músicas com ano maior que 1980 E duração maior que 200 segundos.
3. Selecione título, gênero e ano das músicas do gênero Samba OU Bossa Nova, ordenadas por ano do mais antigo para o mais novo.

Resolução:

-- 1. *Músicas do gênero Rock*

```
SELECT titulo, artista FROM musicas WHERE genero = 'Rock';
```

-- 2. *Ano > 1980 E duração > 200*

```
SELECT * FROM musicas  
WHERE ano > 1980 AND duracao > 200;
```

-- 3. *Samba OU Bossa Nova, ordenadas por ano ASC*

```
SELECT titulo, genero, ano FROM musicas  
WHERE genero = 'Samba' OR genero = 'Bossa Nova'  
ORDER BY ano ASC;
```

Tabela: pedidos

Crie um banco vendas, tabela pedidos, insira os dados e resolva os SELECTs.

Campos:

- id INT
- cliente VARCHAR(60)
- produto VARCHAR(60)
- status VARCHAR(30)
- quantidade INT
- total INT

Registros:

id	cliente	produto	status	quantidade	total
1	Maria Souza	Notebook	Entregue	1	4500
2	João Lima	Smartphone	Enviado	2	6400
3	Ana Clara	Fone Bluetooth	Entregue	3	750
4	Pedro Rocha	Monitor	Pendente	1	1800
5	Carla Dias	Teclado	Enviado	5	2250
6	Lucas Mendes	Tablet	Pendente	2	4200
7	Beatriz Nunes	Smartwatch	Entregue	1	1200
8	Rafael Costa	Carregador	Enviado	10	800

Perguntas (SELECTs):

1. Selecione cliente, produto e total de todos os pedidos com status "Entregue".
2. Selecione todos os pedidos com quantidade maior que 2 E total maior que 1000.
3. Selecione cliente, status e quantidade dos pedidos com status "Enviado" OU "Pendente", ordenados por quantidade do maior para o menor.

Resolução:

-- 1. Pedidos com status Entregue

```
SELECT cliente, produto, total FROM pedidos WHERE status = 'Entregue';
```

-- 2. Quantidade > 2 E total > 1000

```
SELECT * FROM pedidos  
WHERE quantidade > 2 AND total > 1000;
```

-- 3. Enviado OU Pendente, ordenados por quantidade DESC

```
SELECT cliente, status, quantidade FROM pedidos  
WHERE status = 'Enviado' OR status = 'Pendente'  
ORDER BY quantidade DESC;
```